

Module 11 — Microservices (Interview Focus)

High priority for “Java Microservices Developer” roles. You need interview-relevant depth on the Spring Cloud stack: gateway, discovery, config, Feign, resilience, and messaging.

Node.js bridge: Same distributed-systems ideas you know from AWS (API Gateway, service discovery, SQS/SNS, circuit breakers). Here they’re the Spring Cloud + Resilience4j + Kafka ecosystem.

11.1 Monolith vs Microservices

	Monolith	Microservices
Deploy	one unit	many independent services
Scaling	whole app	per-service
Data	one shared DB	DB-per-service
Coupling	tight	loose (network boundaries)
Failure blast radius	whole app	isolated (if resilient)
Complexity	low ops	high ops (network, observability, consistency)

Best answer: “Microservices give independent deployability, per-service scaling, and tech flexibility, at the cost of distributed-systems complexity — network failures, eventual consistency, and observability. I’d start with a well-structured monolith and split out services along clear bounded contexts only when scaling or team autonomy demands it.”

Traps: proposing microservices for a small app; sharing one database across services (defeats the point — each service owns its data).

11.2 API Gateway

- Single entry point for all clients; routes to backend services. Handles cross-cutting concerns: **auth, rate limiting, CORS, request routing, load balancing, aggregation**.
- **Spring Cloud Gateway** (reactive, WebFlux-based; replaced Zuul). Define route predicates + filters.

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE          # lb:// = load-balanced via discovery
          predicates: [ Path=/api/orders/** ]
          filters: [ StripPrefix=2 ]
```

- Benefits: clients don’t know internal topology; centralized security/observability.

11.3 Service Discovery — Eureka

- Services **register** themselves with a registry (**Eureka Server**) and **discover** each other by logical name instead of hardcoded host:port — essential when instances scale up/down dynamically.

- **Internal working:** each instance registers (name + host:port), sends **heartbeats**; Eureka evicts instances that stop heart-beating. Clients cache the registry and load-balance across instances (client-side LB).
- `@EnableEurekaServer` (registry) / `@EnableDiscoveryClient` (services).

```
[order-svc instances] --register/heartbeat--> [Eureka Server] <--fetch registry-- [gateway/clients]
call: use "ORDER-SERVICE" logical name -> resolve to a live instance -> load-balance
```

11.4 Config Server

- **Spring Cloud Config Server** centralizes configuration (backed by a Git repo) for all services and environments; supports refresh without redeploy (`@RefreshScope` + `/actuator/refresh` or Spring Cloud Bus).
- Benefit: one place for config, versioned in Git, per-profile (`service-prod.yml`), secrets via Vault integration.

11.5 Feign Client & Load Balancing

Feign (declarative REST client)

Define an interface; Spring Cloud generates the HTTP client — no boilerplate `RestTemplate/WebClient` code.

```
@FeignClient(name = "ORDER-SERVICE")           // resolves via discovery + load balancer
interface OrderClient {
    @GetMapping("/api/orders/{id}")
    OrderDto getOrder(@PathVariable Long id);
}
```

Load Balancing

- **Client-side LB** (Spring Cloud LoadBalancer, replaced Ribbon): the caller picks an instance from the discovery registry (round-robin by default). `lb://SERVICE-NAME`.
- **Server-side LB** (gateway / cloud LB): a central component distributes requests.

11.6 Circuit Breaker & Resilience4j (key resilience topic)

Why

In a distributed system, one slow/failing downstream can cascade and exhaust threads across the whole system. Resilience patterns contain the failure.

Circuit Breaker states

```
CLOSED --(failure rate > threshold)--> OPEN --(after wait)--> HALF_OPEN
  ^                                     |
  +------(trial calls succeed)-----+
CLOSED      : calls pass through, failures counted
OPEN        : calls fail fast immediately (no downstream call) -> fallback
HALF_OPEN   : allow a few trial calls; success -> CLOSED, failure -> OPEN
```

Resilience4j patterns (lightweight, replaced Hystrix)

- **Circuit Breaker** — stop calling a failing service; fail fast + fallback.
- **Retry** — retry transient failures (with backoff; only for idempotent ops!).
- **Rate Limiter** — cap request rate.
- **Bulkhead** — isolate resources/thread pools so one dependency can't starve others.
- **TimeLimiter** — bound call duration.

```
@CircuitBreaker(name = "orderService", fallbackMethod = "fallback")
@Retry(name = "orderService")
public OrderDto getOrder(Long id) { return orderClient.getOrder(id); }

private OrderDto fallback(Long id, Throwable t) {
    return OrderDto.unavailable(id);           // graceful degradation
}
```

Best Answer

“A circuit breaker prevents cascading failures: after the failure rate crosses a threshold it trips OPEN and fails fast with a fallback instead of hammering a dead service, then probes in HALF_OPEN before closing. I pair it with retries (only for idempotent calls, with backoff), timeouts, and bulkheads via Resilience4j so a single slow dependency can't exhaust the whole service's threads.”

11.7 Distributed Transactions & the Saga Pattern (high level)

The problem

With a **database per service**, you can't use a single ACID transaction across services. Two-phase commit (2PC) is slow and doesn't scale. Solution: **eventual consistency** via Sagas.

Saga Pattern

A distributed transaction = a sequence of **local transactions**, each publishing an event that triggers the next. On failure, run **compensating transactions** to undo prior steps.

```
Order Saga (choreography):
OrderCreated -> [Payment] PaymentDone -> [Inventory] Reserved -> [Shipping] Shipped
If Inventory fails -> emit compensation -> Payment refunded, Order cancelled
```

- **Choreography** — services react to events (decentralized, simple, harder to trace).
- **Orchestration** — a central orchestrator directs steps (clearer, single coordinator).
- Also know **Outbox pattern** (atomic DB write + event) and **idempotent consumers**.

11.8 Kafka & RabbitMQ Basics

Kafka (distributed event log)

- **Topic** split into **partitions** (ordering + parallelism per partition); messages retained (replayable log).
- **Producer** writes; **consumers** in a **consumer group** share partitions (each partition to one consumer in the group → scales horizontally).
- **Offset** tracks read position (consumer-committed). High throughput, durable, replayable.
- Use for: event streaming, high-volume pipelines, event sourcing, decoupling.

RabbitMQ (message broker / queue)

- **Exchange** → **binding** → **queue** routing (direct/topic/fanout). Push-based, per-message ack, smart broker.
- Use for: task queues, RPC, complex routing, lower-volume reliable messaging.

Kafka vs RabbitMQ (guaranteed comparison)

	Kafka	RabbitMQ
Model	distributed log (pull)	queue/broker (push)
Retention	retained, replayable	removed after ack
Throughput	very high	high
Ordering	per partition	per queue
Best for	event streaming, analytics	task queues, routing

Best Answer

“Kafka is a partitioned, replayable log — great for high-throughput event streaming and event sourcing, with ordering per partition and consumer groups for horizontal scaling. RabbitMQ is a smart broker with flexible exchange routing and per-message acks — great for task queues and RPC. I choose Kafka for streaming/high volume and RabbitMQ for complex routing and work queues. For cross-service consistency I use async events with idempotent consumers and the outbox pattern rather than distributed transactions.”

Module 11 — Top 25 Interview Questions (senior answers)

1. **Monolith vs microservices trade-offs?** Independent deploy/scale vs distributed complexity.
2. **When NOT to use microservices?** Small app/team; start monolith, split on need.
3. **Why DB-per-service?** Loose coupling, independent scaling/schema; no shared DB.
4. **What is an API Gateway?** Single entry: routing, auth, rate limit, aggregation.
5. **Spring Cloud Gateway vs Zuul?** Reactive/current vs legacy blocking.
6. **What is service discovery / Eureka?** Dynamic registry; register + heartbeat + discover by name.
7. **Client vs server-side load balancing?** Caller picks instance vs central LB.
8. **What is Feign?** Declarative REST client interface; integrates with discovery + LB.
9. **What is Config Server?** Centralized, Git-backed config across services/envs.
10. **@RefreshScope?** Reload config without redeploy.
11. **Circuit breaker states?** CLOSED → OPEN → HALF_OPEN.
12. **Why circuit breaker?** Prevent cascading failures; fail fast + fallback.
13. **Resilience4j patterns?** CircuitBreaker, Retry, RateLimiter, Bulkhead, TimeLimiter.
14. **Resilience4j vs Hystrix?** Lightweight, functional, active vs deprecated.
15. **When to retry?** Transient failures + idempotent operations, with backoff.
16. **What is a bulkhead?** Resource isolation so one dependency can't starve others.
17. **Distributed transaction problem?** No cross-service ACID; 2PC doesn't scale.
18. **Saga pattern?** Local txns + events + compensating txns for rollback.
19. **Choreography vs orchestration?** Event-reactive decentralized vs central coordinator.
20. **Outbox pattern?** Atomic DB write + event to avoid dual-write inconsistency.
21. **Kafka topic/partition/offset?** Log split for parallelism; consumer-tracked read position.
22. **Consumer group?** Parallel consumption; one partition per consumer in group.
23. **Kafka vs RabbitMQ?** Replayable log/streaming vs broker/queue routing.
24. **How to trace across services?** Correlation id + distributed tracing (Sleuth/Micrometer + Zipkin/OTel).
25. **How to ensure idempotent consumers?** Dedup by message/event id.

Module 11 — Top Coding / Design Questions

- Set up a Feign client with a Resilience4j circuit breaker + fallback.
- Configure a Spring Cloud Gateway route with auth + rate-limit filters.
- Design an order-checkout saga with compensations.
- Design a Kafka producer/consumer for an order-events topic (partitioning + consumer group).
- Add distributed tracing/correlation ids across two services.

Module 11 — Common Follow-ups

- “A downstream service is slow — how do you keep it from taking down your service?” (timeout + circuit breaker + bulkhead.)
- “How do you keep two services’ data consistent without 2PC?” (saga + outbox + idempotency.)
- “Why partition a Kafka topic?” (parallelism + ordering within a key.)

Module 11 — One-Page Cheat Sheet

```
Monolith(simple) vs Microservices(independent deploy/scale, DB-per-service, distributed complexity)
API Gateway: single entry, routing/auth/rate-limit (Spring Cloud Gateway, reactive)
Eureka: register + heartbeat + discover by name; client-side LB (Spring Cloud LoadBalancer)
Feign: declarative REST client (@FeignClient), lb://SERVICE
Config Server: Git-backed central config; @RefreshScope
Resilience4j: CircuitBreaker(CLOSED->OPEN->HALF_OPEN + fallback), Retry(idempotent+backoff),
              RateLimiter, Bulkhead(isolation), TimeLimiter
Distributed tx: no 2PC -> Saga(local txns + compensations), choreography vs orchestration, Outbox,
idempotent consumers
Kafka: partitioned replayable log, consumer groups, offsets -> streaming/high volume
RabbitMQ: exchange->queue routing, per-msg ack -> task queues/routing
Tracing: correlation id + Micrometer/OTel + Zipkin
```

Module 11 — Mock Interview (final module — answer these)

1. “A downstream inventory service starts timing out under load. Walk me through every resilience mechanism you’d apply and why.”
2. “Design order checkout across order, payment, inventory, and shipping services without distributed transactions.”
3. “Explain the circuit breaker state machine and how you’d tune the thresholds.”
4. “Kafka vs RabbitMQ — pick one for an order-events pipeline and justify it.”
5. “How does a request flow from the client through the gateway to a specific service instance?”

Bootcamp Wrap-Up — 5-Day Revision Plan

- **Day 1:** Modules 1–2 (Core Java + Collections). Re-read every ASCII diagram + cheat sheet.
- **Day 2:** Modules 3–5 (Java 8+, Exceptions, Concurrency). Do the coding questions.
- **Day 3:** Modules 6–7 (Spring + Boot). Be able to trace a request end-to-end.
- **Day 4:** Modules 8–9 (JPA + Security). Draw the N+1 fix and the JWT flow from memory.
- **Day 5:** Modules 10–11 (REST + Microservices) + all mock interviews out loud.

Interview-day rules: state the *why*, name the internal mechanism, mention one trap, then give the senior best-answer. Relate to your Node/AWS experience where natural. You’ve got this.